

JavaScript possui estruturas especiais chamadas:

- Map
- Set

Elas ajudam a organizar dados de maneira mais eficiente.

1. SET

Um `Set` é uma coleção de valores únicos.

Ou seja:

ele NÃO permite valores repetidos.

Criando um Set

```
const numeros = new Set();
```

Adicionando valores

Usamos:

```
add()
```

Exemplo:

```
const numeros = new Set();

numeros.add(10);
numeros.add(20);
numeros.add(30);

console.log(numeros);
```

Resultado:

```
Set(3) {10, 20, 30}
```

Valores repetidos

O Set ignora valores repetidos.

```
const numeros = new Set();

numeros.add(10);
numeros.add(10);
numeros.add(10);

console.log(numeros);
```

Resultado:

```
Set(1) {10}
```

Mesmo adicionando várias vezes:

- só fica um valor.
-

Visualizando mentalmente

```
10
10
10
```

O Set pensa:

| "esse valor já existe"

Então ele não adiciona novamente.

Verificando se existe

Usamos:

```
has()
```

Exemplo:

```
const nomes = new Set();  
  
nomes.add("Gabriel");  
  
console.log(nomes.has("Gabriel"));
```

Resultado:

```
true
```

Removendo valores

Usamos:

```
delete()
```

Exemplo:

```
const nomes = new Set();  
  
nomes.add("João");  
  
nomes.delete("João");  
  
console.log(nomes);
```

Resultado:

```
Set(0) {}
```

Tamanho do Set

Usamos:

```
size
```

Exemplo:

```
const numeros = new Set();

numeros.add(1);
numeros.add(2);

console.log(numeros.size);
```

Resultado:

```
2
```

Percorrendo um Set

```
const frutas = new Set();

frutas.add("Maçã");
frutas.add("Banana");
frutas.add("Uva");

for (const fruta of frutas) {

    console.log(fruta);

}
```

Resultado:

```
Maçã
Banana
Uva
```

Onde Set é útil?

Muito usado para:

- remover duplicados
 - listas únicas
 - controle de valores repetidos
-

Exemplo removendo duplicados

```
const numeros = [1,1,2,2,3,3];  
  
const unico = new Set(numeros);  
  
console.log(unico);
```

Resultado:

```
Set(3) {1, 2, 3}
```

2. MAP

O Map funciona como um objeto.

Mas ele guarda:

- chave
 - valor
-

Criando um Map

```
const pessoa = new Map();
```

Adicionando valores

Usamos:

```
set()
```

Exemplo:

```
const pessoa = new Map();

pessoa.set("nome", "Gabriel");
pessoa.set("idade", 17);

console.log(pessoa);
```

Resultado:

```
Map(2) {
  'nome' => 'Gabriel',
  'idade' => 17
}
```

Visualização mental

```
nome → Gabriel
idade → 17
```

Pegando valores

Usamos:

```
get()
```

Exemplo:

```
console.log(pessoa.get("nome"));
```

Resultado:

```
Gabriel
```

Verificando se existe

Usamos:

```
has()
```

Exemplo:

```
console.log(pessoa.has("idade"));
```

Resultado:

```
true
```

Removendo valores

Usamos:

```
delete()
```

Exemplo:

```
pessoa.delete("idade");
```

Quantidade de itens

Usamos:

```
size
```

Exemplo:

```
console.log(pessoa.size);
```

Percorrendo Map

```
const pessoa = new Map();

pessoa.set("nome", "Gabriel");
pessoa.set("idade", 17);

for (const [chave, valor] of pessoa) {

    console.log(chave, valor);

}
```

Resultado:

```
nome Gabriel
idade 17
```

Diferença entre Object e Map

Object	Map
mais simples	mais poderoso
chaves geralmente texto	aceita qualquer tipo
menos recursos	mais métodos

Diferença entre Set e Map

Set	Map
guarda valores únicos	guarda chave e valor
não repete valores	usa pares
foco em unicidade	foco em organização

Resumo rápido

Estrutura	Serve para
Set	valores únicos
Map	chave e valor

Métodos importantes do Set

Método	Função
add()	adiciona
has()	verifica
delete()	remove
size	quantidade

Métodos importantes do Map

Método	Função
set()	adiciona
get()	pega valor
has()	verifica
delete()	remove
size	quantidade

Ideia principal

Set

"não quero valores repetidos"

Map

"quero organizar dados usando chave e valor"